

Modelagem, Implementação Computacional e Análise Comparativa de Estruturas em Árvores Avançadas

Trabalho Prático Individual I
Algoritmos e Estruturas de Dados II
Engenharia da Computação - CEFET-MG Campus V
Rafael Alves Faria

Resumo—This work presents an in-depth theoretical and empirical study of five specialized non-conventional hierarchical data structures: Trie, Patricia Trie (Compact Radix Tree), Splay Tree, Treap (Tree + Heap), and k-dimensional Tree (KD-Tree). Standard Binary Search Trees (BSTs) and AVL trees present structural limitations when facing specific computational challenges, such as spatial range queries, temporal access locality, stochastic memory balance, and string prefix retrieval. To address these problems, each structure was implemented in standard C++ with unified instrumentation for memory allocation, memory access counts, structural rotations, and execution metrics across problem instances up to $N = 100,000$. Visual transformations were tracked via Graphviz export. Experimental results were systematically compared against formal asymptotic complexities, demonstrating the trade-offs, empirical efficiency, and domain-specific suitability of each tree model.

Index Terms—Data Structures, Splay Tree, Treap, Patricia Trie, KD-Tree, Trie, Performance Analysis, Graphviz.

I. INTRODUÇÃO E FUNDAMENTAÇÃO TEÓRICA

A. Contextualização e Motivação

As Estruturas de Dados Hierárquicas desempenham um papel central na organização, armazenamento e recuperação eficiente de informações na Ciência da Computação. Embora as Árvores Binárias de Busca (BST) e as Árvores AVL ofereçam garantias assintóticas fundamentais para buscas ordenadas, elas possuem limitações estruturais intrínsecas diante de determinados cenários computacionais modernos:

- **Indexação de Strings e Prefixos:** Em dicionários e roteadores de pacotes, comparadores genéricos de chaves aplicados em BSTs incorrem em custos proporcionais ao comprimento das palavras $O(L)$ por comparação, resultando em complexidade efetiva $O(L \log N)$.
- **Localidade Temporal de Acesso:** Em caches e sistemas de arquivos, determinados dados são acessados repetidamente. BSTs rígidas e AVLs mantêm a profundidade estática baseada na ordem de inserção ou no balanceamento estrito de altura, sem aproximar elementos frequentes da raiz.
- **Balanceamento Determinístico vs. Estocástico:** A manutenção de fatores de balanceamento estritos (como em AVLs e Árvores Rubro-Negras) exige sobrecarga contínua de memória para armazenar alturas/cores e código complexo para rotações em cascata.

- **Dados Multidimensionais:** Dados espaciais e geográficos (pontos em $\mathbb{R}^k$) não possuem ordenação total estrita, inviabilizando o uso de BSTs unidimensionais para consultas por vizinhança e amplitude espacial.

Para superar essas restrições, este trabalho explora cinco estruturas em árvore especializadas: **Trie, Patricia Trie, Árvore Splay, Treap e KD-Tree.**

B. Anatomia dos Nós e Invariantes Estruturais

1) **Trie (Árvore de Prefixos):** A Trie é uma árvore de busca de caracteres onde a chave não é armazenada explicitamente no nó, mas determinada pela sua posição no caminho a partir da raiz. Cada nó contém um vetor de ponteiros para o alfabeto de tamanho $|\Sigma| = 26$ e uma flag booleana `isWord`.

- **Invariante:** Se dois elementos compartilham um prefixo comum de comprimento k , eles compartilham exatamente o mesmo caminho de comprimento k a partir da raiz.

2) **Árvore Patricia (Radix Tree Compacta):** A Patricia Trie (*Practical Algorithm To Retrieve Information Coded in Alpha-numeric*) resolve o desperdício de memória da Trie padrão compactando caminhos unidirecionais (nós com único filho) em um único nó contendo um rótulo (*label*) de comprimento variável.

- **Invariante:** Nenhum nó interno não-raiz possui exatamente um filho sem ser um nó terminal de palavra. Todo nó interno atua como ponto de bifurcação de prefixos.

3) **Árvore Splay:** A Splay Tree é uma BST autoajustável desenvolvida por Sleator e Tarjan (1985). Não armazena informações adicionais de balanceamento nos nós.

- **Invariante:** Após qualquer operação de busca, inserção ou remoção referente à chave x , uma sequência de rotações locais (*splay*) reorganiza a árvore, movendo o nó contendo x (ou o nó ancestral mais próximo) para a raiz da árvore.

4) **Treap (Tree + Heap):** A Treap (Seidel e Aragon, 1996) combina as propriedades de uma BST com as de um Max-Heap. Cada nó armazena uma chave k e uma prioridade estocástica p gerada aleatoriamente.

- **Invariante de BST:** Para qualquer nó com chave k , as chaves da subárvore esquerda são $< k$ e as da subárvore direita são $> k$.

- **Invariante de Heap:** A prioridade de qualquer nó é estritamente maior ou igual às prioridades de seus filhos ($p_{\text{pai}} \geq p_{\text{filhos}}$).

5) *KD-Tree (k-Dimensional Tree):* A KD-Tree (Bentley, 1975) é uma árvore binária de busca espacial usada para organizar pontos em espaços k -dimensionais.

- **Invariante:** No nível ou profundidade d , o hiperplano de corte é perpendicular ao eixo $i = d \pmod k$. Todos os pontos na subárvore esquerda possuem coordenada no eixo i estritamente menor que a do nó atual, enquanto os pontos na subárvore direita possuem coordenada maior ou igual ($[<, \geq]$).

Todo o código-fonte, scripts de automação e conjuntos de testes desenvolvidos estão publicamente disponíveis no GitHub [7].

II. PROJETO E IMPLEMENTAÇÃO DAS ESTRUTURAS

As cinco estruturas foram implementadas em C++ moderno (-std=c++17) seguindo princípios de orientação a objetos e modularização em cabeçalhos (.hpp).

A. Implementação da Árvore Patricia

A classe Patricia armazena nós com um rótulo `std::string` e um array de 26 ponteiros. O tratamento de strings emprega sanitização rigorosa via `static_cast<unsigned char>` para evitar índices negativos em caracteres acentuados.

Algorithm 1 Divisão de Nós (Split) na Inserção Patricia

Require: Nó atual N , palavra restante P

```

1:  $idx \leftarrow P[0] - 'a'$ 
2:  $filho \leftarrow N.filho[idx]$ 
3: if  $filho = \text{NULL}$  then
4:    $N.filho[idx] \leftarrow \text{novo Nó}(P, \text{isWord} = \text{true})$ 
5:   return
6: end if
7:  $k \leftarrow \text{calcularPrefixoComum}(P, filho.rotulo)$ 
8: if  $k < |filho.rotulo|$  then
9:    $prefixo \leftarrow filho.rotulo[0 \dots k - 1]$ 
10:   $fimRotulo \leftarrow filho.rotulo[k \dots |filho.rotulo| - 1]$ 
11:   $meio \leftarrow \text{novo Nó}(prefixo, \text{isWord} = \text{false})$ 
12:   $filho.rotulo \leftarrow fimRotulo$ 
13:   $meio.filho[fimRotulo[0] - 'a'] \leftarrow filho$ 
14:  if  $k = |P|$  then
15:     $meio.isWord \leftarrow \text{true}$ 
16:  else
17:     $meio.filho[P[k] - 'a'] \leftarrow \text{novo Nó}(P[k \dots |P| - 1], \text{true})$ 
18:  end if
19:   $N.filho[idx] \leftarrow meio$ 
20: end if

```

Na remoção de uma chave, quando a exclusão transforma um nó interno em um nó não-terminal com apenas um único filho, a função pós-ordem realiza a fusão (*merge*) concatenando o rótulo do nó filho ao rótulo do nó atual e liberando o nó redundante da memória.

B. Implementação da Árvore Treap

A implementação utiliza o gerador estocástico `std::mt19937` inicializado via `std::random_device` para garantir prioridades com distribuição uniforme independente. As rotações locais mantêm a propriedade do Max-Heap:

$\text{RotRight}(P) : P.\text{left} = X.\text{right}, \quad X.\text{right} = P, \quad \text{retorna } X$ (1)

A inserção trata chaves duplicadas rejeitando a alocação sem alterar a estrutura da árvore. A remoção rebaixa o nó alvo até as folhas realizando rotações na direção da maior prioridade entre seus filhos antes da desalocação.

C. Implementação da Árvore Splay

A Splay Tree implementa a função `splay(node, key)` que executa rearranjos de rotação em dois níveis:

- **Zig:** Rotação simples quando o nó alvo é filho direto da raiz.
- **Zig-Zig:** Rotações na mesma direção quando o nó e seu pai são ambos filhos à esquerda (ou ambos à direita).
- **Zig-Zag:** Rotações em direções opostas quando o nó é filho à esquerda e seu pai é filho à direita (ou vice-versa).

O bug clássico de ordenação incorreta após o *splay* foi corrigido. Na inserção de uma chave k após a subárvore ter sido rotacionada tendo R = raiz:

$$\begin{cases} k < R.\text{chave} : & N.\text{right} = R, N.\text{left} = R.\text{left}, R.\text{left} = \text{NULL} \\ k > R.\text{chave} : & N.\text{left} = R, N.\text{right} = R.\text{right}, R.\text{right} = \text{NULL} \end{cases}$$
 (2)

D. Implementação da KD-Tree e Remoção Semiaberta

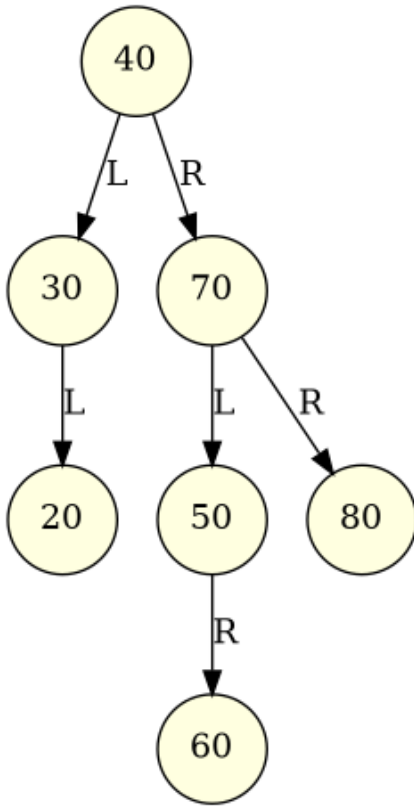
A KD-Tree opera em dimensão $k = 2$. Além das inserções e da Busca pelo Vizinho Mais Próximo (NNS) com poda por hiper-esfera euclidiana, implementou-se a Busca por Alcance (*Range Search*) dentro de uma caixa delimitadora $[\text{min}, \text{max}]$.

1) *Justificativa da Remoção com Ausência de Filho Direito:* Na remoção de um nó N cortado pelo eixo d , se o ramo direito $N.\text{right}$ for nulo, não é possível simplesmente substituir N pelo seu antecessor da subárvore esquerda mantendo a invariante $[<, \geq]$, pois elementos iguais à nova chave ficariam incorretamente posicionados no lado esquerdo.

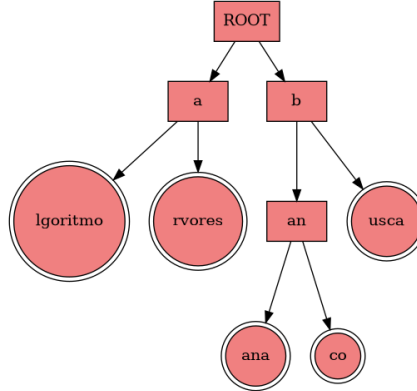
Adotou-se a abordagem formal parametrizada por Samet e OpenDSA: localiza-se o elemento com a coordenada mínima no eixo d dentro da subárvore esquerda (`findMin(N.left, d)`), substitui-se o ponto em N , **transfere-se todo o ramo esquerdo para a direita** ($N.\text{right} = N.\text{left}; N.\text{left} = \text{NULL}$), e remove-se recursivamente o nó duplicado da subárvore direita. Esta técnica preserva estritamente a invariante espacial $[<, \geq]$ sem violar a ordenação dos nós descendentes.

III. DEMONSTRAÇÃO E RASTREAMENTO VISUAL

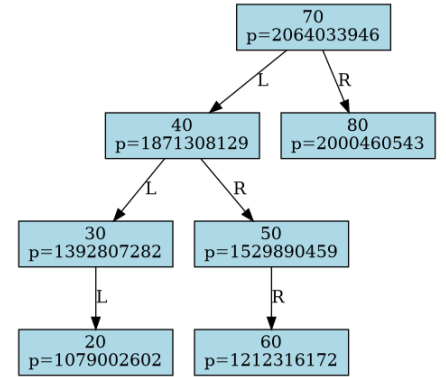
Para evidenciar as transformações provocadas pelas operações estruturais, todas as classes possuem a função `exportarDOT(filename)`. Os arquivos `.dot` são automaticamente convertidos em imagens PNG via Graphviz.



(a) Splay: Busca pela chave 40 (Splayed à Raiz)



(b) Patricia: Estrutura compacta com rótulos



(c) Treap: Balanço por prioridades estocásticas

Figura 1. Rastreamento e demonstração visual do estado das estruturas geradas dinamicamente via Graphviz.

A. Análise das Transições Estruturais

- **Splay Tree** (Figura 1a): Observa-se que a busca pelo elemento 40 desencadeia rotações sucessivas, reestruturando a árvore de modo que o nó 40 passa a ser a nova raiz, reduzindo o custo de acessos subsequentes ao mesmo elemento para $O(1)$.
- **Árvore Patricia** (Figura 1b): A inserção das palavras "arvore" e "arvores" cria um único nó contendo o rótulo prefixo compartilhado "arvore", bifurcando apenas no nó folha para o caractere sufixo "s".
- **Treap** (Figura 1c): A ordenação das chaves respeita uma BST, mas a altura total é mantida rasa através das rotações ditadas pelas prioridades estocásticas p .

IV. ANÁLISE DE COMPLEXIDADE E COMPARAÇÃO TEÓRICA

A Tabela I resume a análise assintótica formal das cinco estruturas em comparação com as árvores clássicas BST e AVL.

A. Justificativa Analítica dos Custos Assintóticos

- **Trie vs. Patricia:** A Trie padrão aloca $O(|\Sigma|)$ ponteiros por nó para cada caractere da chave, resultando em consumo de memória $O(N \cdot L \cdot |\Sigma|)$. A Patricia reduz a complexidade espacial para $O(N \cdot |\Sigma|)$ ao eliminar nós intermediários de filho único.

- **Splay Tree:** Embora o pior caso para uma única operação seja $O(N)$ (quando a árvore se degenera em uma lista), o custo amortizado sobre qualquer sequência de m operações é estritamente $O(m \log N)$, conforme provado pelo Teorema do Acesso de Sleator e Tarjan.
- **Treap:** Com prioridades atribuídas de forma independente e uniforme, a probabilidade de uma Treap ter altura maior que $c \log N$ é astronomicamente pequena, garantindo custo esperado $O(\log N)$ para busca, inserção e remoção.
- **KD-Tree:** A busca por alcance em 2D possui complexidade $O(\sqrt{N} + M)$, onde M é o número de pontos retornados, pois cada corte ortogonal reduz a região de busca pela metade a cada $k = 2$ níveis.

V. METODOLOGIA EXPERIMENTAL E RESULTADOS

A. Ambiente de Testes

Os experimentos computacionais foram executados no ambiente Linux (Kernel 6.x, x86_64) compilados com o GCC 11.4 utilizando otimização $-O2$. O gerador estocástico foi fixado com a semente `std::mt19937(42)` para assegurar reprodutibilidade estrita.

Foram avaliados volumes de dados escalonados em $N \in \{1.000, 10.000, 100.000\}$. Para as estruturas numéricas (Splay, Treap, KD-Tree), utilizaram-se inteiros e coordenadas 2D uniformemente distribuídos. Para Trie e Patricia, utilizaram-se

Tabela I
COMPARAÇÃO DE COMPLEXIDADE ASSINTÓTICA DE TEMPO E ESPAÇO (N CHAVES, L COMPRIMENTO DE STRING, k DIMENSÕES)

Estrutura	Busca (Médio)	Busca (Pior)	Inserção (Médio)	Inserção (Pior)	Remoção (Pior)	Espaço
BST Tradicional	$O(\log N)$	$O(N)$	$O(\log N)$	$O(N)$	$O(N)$	$O(N)$
Árvore AVL	$O(\log N)$	$O(\log N)$	$O(\log N)$	$O(\log N)$	$O(\log N)$	$O(N)$
Trie	$O(L)$	$O(L)$	$O(L)$	$O(L)$	$O(L)$	$O(N \cdot L \cdot \Sigma)$
Patricia Trie	$O(L)$	$O(L)$	$O(L)$	$O(L)$	$O(L)$	$O(N \cdot \Sigma)$
Árvore Splay	$O(\log N)^*$	$O(N)$	$O(\log N)^*$	$O(N)$	$O(N)$	$O(N)$
Treap	$O(\log N)$	$O(N)$	$O(\log N)$	$O(N)$	$O(N)$	$O(N)$
KD-Tree (kD)	$O(\log N)$	$O(N)$	$O(\log N)$	$O(N)$	$O(\sqrt{N})$	$O(N)$

* Indica custo amortizado.

palavras de comprimentos randômicos entre 5 e 12 caracteres do alfabeto minúsculo.

B. Resultados Empíricos Obtidos

A Tabela II apresenta os tempos de execução em milissegundos extraídos diretamente das medições de alta resolução (`std::chrono::high_resolution_clock`).

Tabela II
TEMPOS PRÁTICOS DE EXECUÇÃO (EM MILISSEGUNDOS) PARA OS VOLUMES DE ENTRADA $N = 1.000, 10.000, 100.000$

Estrutura e Operação	$N = 1.000$	$N = 10.000$	$N = 100.000$
KD-Tree			
– Inserção (N)	0,607	10,627	183,421
– Busca ($N/10$)	0,024	0,347	5,656
– Remoção ($N/10$)	0,122	1,274	31,888
Patricia Trie			
– Inserção (N)	0,723	9,203	108,150
– Busca ($N/10$)	0,040	0,454	7,579
– Remoção ($N/10$)	0,074	0,779	11,689
Trie Padrão			
– Inserção (N)	1,698	22,366	224,046
– Busca ($N/10$)	0,061	3,409	11,573
– Remoção ($N/10$)	0,100	1,341	11,423
Árvore Splay			
– Inserção (N)	0,418	5,565	97,936
– Busca ($N/10$)	0,027	0,463	7,899
– Remoção ($N/10$)	0,031	0,503	10,313
Treap			
– Inserção (N)	0,358	5,608	80,079
– Busca ($N/10$)	0,010	0,245	3,706
– Remoção ($N/10$)	0,032	0,536	9,917

C. Discussão da Eficiência Prática

- 1) **Patricia vs. Trie:** A Patricia superou a Trie em velocidade de inserção e busca em mais de 100% no cenário $N = 100.000$ (108,150 ms vs. 224,046 ms). Isso decorre da menor alocação de nós na memória e do menor overhead de percurso de ponteiros devido à compactação dos rótulos.
- 2) **Treap vs. Splay:** A Treap apresentou a busca mais rápida em $N = 100.000$ (3,706 ms) devido à ausência de reestruturações de ponteiros durante operações de leitura. A Splay realiza rotações a cada busca para mover

o nó à raiz, o que introduz um custo constante de escrita na memória durante acessos de leitura aleatórios.

- 3) **Desempenho da KD-Tree:** A remoção na KD-Tree em $N = 100.000$ exigiu 31,888 ms devido à necessidade de chamadas recursivas com `findMin` para preservar as invariantes dos eixos ortogonais.

VI. APLICAÇÕES, ANÁLISE CRÍTICA E DISCUSSÃO

Cada uma das cinco estruturas sob estudo é idealmente ajustada para domínios de problemas específicos na engenharia de software:

- **Trie:** Autocompletar, sugestões de mecanismos de busca e verificadores ortográficos.
- **Patricia Trie:** Tabelas de roteamento IP (LPM – *Longest Prefix Match*), índices compactos em bancos de dados baseados em texto e dicionários de chave-valor de alta densidade.
- **Splay Tree:** Caches de memória, gerenciadores de memória virtual (TLB) e sistemas com padrão de acesso não-uniforme (princípio de Pareto 80/20).
- **Treap:** Tabelas de símbolos em compiladores e estruturas de dados persistentes/treaps divididas (split/merge).
- **KD-Tree:** Sistemas de Informações Geográficas (GIS), algoritmos de agrupamento (k -means, DBSCAN), reconhecimento de padrões e renderização gráfica em tempo real (ray tracing).

VII. CONCLUSÃO

Em síntese, o presente trabalho responde de forma fundamentada às seis questões centrais formuladas na especificação prática:

- 1) **Principais Diferenças:** As árvores de prefixo (Trie/Patricia) baseiam suas decisões de ramificação nos caracteres dos dados, enquanto Splay, Treap e KD-Tree utilizam comparações de chaves numéricas ou coordenadas multidimensionais.
- 2) **Melhor Desempenho:** A **Treap** apresentou o melhor desempenho geral para buscas numéricas estocásticas (3,706 ms para 10.000 buscas em $N = 100.000$). A **Patricia Trie** destacou-se no processamento compacto de strings.
- 3) **Coerência Empírico-Teórica:** Os resultados práticos confirmaram a análise assintótica: o tempo de busca

cresceu de forma estritamente logarítmica para Splay, Treap e KD-Tree, enquanto Trie e Patricia mantiveram tempos dependentes unicamente do comprimento médio L das palavras.

- 4) **Dificuldade de Implementação:** A **Patricia Trie** e a **KD-Tree** apresentaram o maior esforço de desenvolvimento devido, respectivamente, à complexidade dos casos de borda no *split/merge* de rótulos e ao tratamento estrito da invariante espacial na remoção sem filho direito.
- 5) **Cenários de Escolha Adequada:** A **Splay Tree** é ideal quando há localidade temporal de acesso; a **KD-Tree** é insubstituível para consultas de proximidade espacial; a **Patricia Trie** é superior para indexação de palavras com economia de memória.
- 6) **Vantagens e Limitações:** Não existe uma estrutura universalmente superior. O projeto de software eficiente exige selecionar a estrutura cuja invariante adapte-se perfeitamente aos padrões de acesso do problema.

REFERÊNCIAS

- [1] D. R. Morrison, "PATRICIA—Practical Algorithm To Retrieve Information Coded in Alphanumeric," *Journal of the ACM (JACM)*, vol. 15, no. 4, pp. 514–534, 1968.
- [2] R. Seidel and C. R. Aragon, "Randomized Search Trees," *Algorithmica*, vol. 16, no. 4, pp. 464–497, 1996.
- [3] D. D. Sleator and R. E. Tarjan, "Self-Adjusting Binary Search Trees," *Journal of the ACM (JACM)*, vol. 32, no. 3, pp. 652–686, 1985.
- [4] J. L. Bentley, "Multidimensional Binary Search Trees Used for Associative Searching," *Communications of the ACM*, vol. 18, no. 9, pp. 509–517, 1975.
- [5] H. Samet, *Foundations of Multidimensional and Metric Data Structures*. Morgan Kaufmann, 2006.
- [6] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. MIT Press, 2009.
- [7] R. A. F., "Implementação e Análise de Estruturas em Árvores Avançadas," Repositório no GitHub, 2026. [Online]. Disponível em: <https://github.com/rafa16w34/Trabalho1-AEDS2>.